Lab 4

1-You are tasked with designing an **Employee Management System** that models a simple organizational hierarchy.

The system should include the following classes:

1. Person Class (Base Class)

- Represents a general person with the following attributes:
    - name (String) – The name of the person.
    - age (int) – The age of the person.
- Has a constructor to initialize the attributes.
- Implements a method displayInfo() that prints the **name** and **age** of the person.

2. Employee Class (Inherits from Person)

- Represents an employee with additional attributes:
    - employeeId (String) – The employee's unique ID.
    - salary (double) – The employee's base salary.
- Has a constructor that initializes **all** attributes.
- Implements a method calculateBonus() that calculates the employee's **bonus as 10% of their salary**.
- Overrides displayInfo() to include employee-specific details and the bonus.

3. Manager Class (Inherits from Employee)

- Represents a manager with additional attributes:
    - department (String) – The department the manager leads.
    - extraBonus (double) – A fixed additional bonus for managers.
- Has a constructor that initializes **all** attributes.
- Implements a modified calculateBonus() method, where the **bonus is calculated based on salary, age, and extra bonus**. "As (salary * 0.20) + (age * 50) + extraBonus"

- Overrides displayInfo() to include manager-specific detail.

Requirements

- The program should allow creating **Employee** and **Manager** objects.
- The displayInfo() method should provide all relevant details for each role.
- The **bonus calculation** should be handled differently for employees and managers.

Example test code

```
public static void main(String[] args) {
    System.out.println("=== Employee Info ===");
    Employee employee = new Employee("Bob", 25, "E123", 50000);
    employee.displayInfo();

    System.out.println("\n=== Manager Info ===");
    Manager manager = new Manager("Charlie", 40, "M456", 80000, "IT", 5000);
    manager.displayInfo();
}
```

Expected output

```
=== Employee Info ===
Name: Bob
Age: 25
Employee ID: E123
Salary: 50000.0
Bonus: 5000.0

=== Manager Info ===
Name: Charlie
Age: 40
Employee ID: M456
Salary: 80000.0
Bonus: 23000.0
Department: IT
```

2- Provided is a custom-made String class called **MyStringBuffer**. Internally, the class has a private reference to an array of characters allocated at construction time (based on a passed String). Additionally, it has all the following methods:

• void print(): Prints the current String.

• int indexOf(char c): Returns the index of the specified character in the current object or -1 if not found.

• int length(): Returns the length of the string

• void erase(int index, int length): Modifies the current object by erasing a substring whose position and length are specified by the passed values.

• void insert(int index, String str): Modifies the current object by inserting the specified string at the specified position.

Answer the following:

1. Is the provided class Immutable?
2. If not immutable, why?
3. If not immutable, change the code to make it immutable while preserving the functionality of the class.

```
public class MyStringBuffer {
    private char[] str;

    public MyStringBuffer(String s) {
        this.str = new char[s.length()];
        for (int i = 0; i < s.length(); i++) {
            this.str[i] = s.charAt(i);
        }
    }
    public void print() {
        for (int i = 0; i < this.str.length; i++) {
            System.out.print(str[i]);
        }
        System.out.println("");
    }
```

```
public int indexOf(char c) {
        for (int i = 0; i < str.length; i++) {
                if (c == str[i]) {
                        return i;
                }
        }
        return -1;
}
public int length() {
        return str.length;
}
public void erase(int index, int length) {
        //Extra: Check that index must be >0 and < str.length
        if (!(index >= 0 && index < str.length)) {
                System.out.println("Couldn't erase index is incorrect");
                return;
        }
        char[] str2;
        if (index + length >= str.length) {
                str2 = new char[index];
                for (int i = 0; i < index; i++) {
                        str2[i] = this.str[i];
                }
        } else {
                str2 = new char[this.str.length - length];
                int j = 0;
                for (int i = 0; i < index; i++) {
```

```
                    str2[j++] = this.str[i];
                }
                for (int i = index + length; i < str.length; i++) {
                    str2[j++] = this.str[i];
                }
            }
            this.str = str2;
        }
    }
```